

Road Lane Overlay: Generative Restoration of Lane Markings in Dashcam Footage

Arunesh Mishra Mrunmay Deshmukh Prachi Dudhe Shreya Muppidi

Texas A&M University

aruneshm@tamu.edu mrunmayd@tamu.edu pra24@tamu.edu shreyamuppidi@tamu.edu

Abstract

Reliable perception of lane markings is essential for driver assistance and autonomous driving, yet motion blur, worn paint, glare, rain/fog, and night scenes often erase the thin, high-frequency lane lines that planners rely on. We propose a lane-aware restore $\rightarrow$ detect $\rightarrow$ repair pipeline that edits only where lane evidence is missing. First, an optional visibility boost stage applies a Dark Channel Prior-based dehazing/deraining module to improve contrast on thin paint while preserving road structure. Next, CLRerNet is run on the cleaned frame to obtain confidence-calibrated lane polylines, which are expanded into narrow lane-band masks (6–12 px) and further reduced to sparse gap masks marking only broken or weak segments. We initially explored LaMa for blind inpainting inside these regions but found that it consistently extended surrounding asphalt texture instead of reconstructing thin, high-contrast markings. To address this, we fine-tune the Qwen-Image-Edit model using Green CLRerNet overlays as control images and text prompts that explicitly request realistic white/yellow lane repainting while preserving other scene content. On CULane, fine-tuning improves FID by 12.3% (34.54 $\rightarrow$ 30.28) and lane F1 by 7.1%, with a 10.3% gain in recall, demonstrating that vision-language-guided generative restoration can recover lane markings under adverse conditions without degrading overall image fidelity.

1. Introduction

1.1. Background and Problem Setting

Modern driver assistance and autonomous stacks depend on camera perception to localize lanes and recognize regulatory or warning signs. In real driving, motion blur, worn paint, headlight glare, precipitation, and low light degrade exactly the thin structures that detectors rely on. Models trained on clean images often output fragmented lanes, miss small symbols, or fire on ambiguous markings.

Generative methods offer a complementary capability: restore image evidence before detection or train restoration jointly with detection. Prior studies report higher lane accuracy under blur when an enhancement module is included. Broader transportation surveys also show that generative models help with data sparsity, uncertainty, and rare scenes. Practical deployment, however, requires compact designs that sustain near real-time throughput.

Scope: We work on still frames (images) from standard dash-cam dataset (e.g., CULane) and target lane restoration under adverse conditions, with edits to thin lane bands (6–12 px)

1.2. Motivation

On-board perception breaks down when lane paint is faded, scenes are blurred, or weather and lighting are poor. Thin lane lines disappear first, which causes broken tracks and unstable planning. Many cameras on real cars and dash-cams face these conditions daily, so a simple detector alone is not enough. A practical pipeline that first restores visibility, then finds lanes, and finally fills only the missing pieces can keep lane guidance stable without rewriting the whole frame.

1.3. Research Gap

Most lane systems try to detect directly on degraded frames, or they enhance the whole image at once. Global enhancement can create artifacts, change road appearance, and still miss thin lines. Existing inpainting often edits large areas and is not tied to lane geometry. There is limited work that:

1. Clears fog, rain, and low light first to recover contrast,
2. Detects lanes next to localize gaps precisely, and
3. Runs guided inpainting only on those tiny gaps using simple structure rules such as lane width, edge continuity, and road surface constraints.

This gap motivates a restore-then-detect-then-repair approach that edits only where lanes are missing, preserves the rest of the scene, and prioritizes continuity of lane markings for stable planning.

2. Related Work

Recent research in autonomous driving has increasingly focused on enhancing road layout understanding through generative, attention-based, and restoration-driven techniques. Traditional lane detection pipelines struggle under occlusion, poor visibility, and degraded markings, prompting a shift toward semantic and generative frameworks that can infer missing or unclear lane structures. This section reviews key works that inform and align with our project on road lane overlay detection.

2.1. Improving Confidence of Lane Detection With LaneIoU

CLRerNet[7] builds on CLRNet by introducing LaneIoU, a supervision strategy that aligns detection confidence with the IoU between predicted and ground truth lanes. This improves confidence calibration and overall performance. The model uses structured, learnable anchors $(x_a, y_a, \theta_a, \ell)$ and optimizes a combined loss of regression, classification, segmentation, and LaneIoU terms. Trained on CULane with 30-pixel binary masks and 5-fold CV, CLRerNet achieves $F1@0.5 = 81.43\%$ and $F1@0.75 = 64.28\%$, improving over CLRNet and reaching 86.47% on CurveLanes. However, a large gap from the oracle score (98.47%) highlights remaining challenges in confidence calibration and noise sensitivity. CLRerNet provides a strong baseline for lane detection, suitable for downstream masking or restoration tasks.

2.2. Keep Your Eyes on the Lane (ALDNet)

ALDNet[14] introduces attention-guided mechanisms to focus feature learning on lane-relevant regions. Its lightweight spatial attention module enhances robustness under complex conditions like low light, rain, or occlusion while maintaining real-time inference speed. Tested on TuSimple and CULane, ALDNet achieves state-of-the-art accuracy, particularly under adverse visual scenarios. Through ablation and visualization studies, the authors demonstrate that attention improves interpretability and localization, an essential trait for safety-critical perception modules in autonomous vehicles.

2.3. Resolution-Robust Large Mask Inpainting With Fourier Convolutions

LaMa[13] proposes an efficient inpainting model using Fast Fourier Convolutions (FFCs) to capture global context early and a high receptive field perceptual loss for sharper restoration. A ResNet-like generator processes a masked image concatenated with its binary mask, trained with a multi-term objective combining adversarial, perceptual, feature-matching, and R1 regularization losses. On Places and CelebA-HQ, LaMa achieves competitive results $FID = 0.63$, $LPIPS = 0.090$ (narrow masks) and $FID = 2.21$, $LPIPS = 0.135$ (wide masks) while maintaining a simpler training

setup. Although it struggles with highly distorted or unseen structures, LaMa is effective for generating visually consistent lane restorations.

2.4. Lane Occlusion Inpainting and Detection

LOID[1] proposes a restore-then-detect pipeline that identifies occluders, inpaints missing regions, and then segments lanes. The approach yields significant performance gains 20–24% mIoU improvement on CULane and BDD100K under occluded conditions while retaining near real-time speed. The authors also find that augmenting data with synthetic occluders helps but remains less effective than generative completion. This study underscores that lane detectors require visually consistent reconstructions before segmentation, validating the principle behind our overlay approach.

2.5. Visual language models in autonomous driving (Soininen, 2024)

We referenced Soininen’s[12] thesis because it demonstrates, with concrete experiments, how pairing Visual Language Models with Large Language Models can lift perception from raw detections to situation aware reasoning that produces actionable control guidance, a capability we need when lane cues are weak or missing. The study compares Qwen VL, CogVLM, and DeepSeek VL across obstacle localization and scene explanation tasks, while the strongest gains come from the second track where DeepSeek VL consistently generates coherent situation descriptions and control actions detailed enough to be turned into executable Python functions. In our pipeline, we see a natural hand-off from confidence calibrated and attention guided lane extraction to language based decision scaffolding that can infer safe maneuvers under occlusion, rain, glare, or degraded markings, and surface interpretable narratives for downstream controllers. Our takeaway is that multimodal reasoning complements geometric lane extraction by adding context, intent, and control hints that enhance robustness in ambiguous scenes.

2.6. A Lightweight Vision–Language Model for Cross-Resolution GUI Grounding

The authors show that careful dataset design and a two stage fine tuning pipeline produce a model that rivals much larger architectures on GUI grounding benchmarks[8]. Key technical choices include multi resolution data augmentation to improve scale invariance and a grounding objective that maps natural language queries to spatial targets. The paper emphasizes practical constraints and trade offs: by prioritizing parameter efficiency and training feasibility. For our lane overlay detection this work is instructive in three concrete ways. First, the multi resolution training strategy suggests a way to make lane grounding robust to camera zoom. Second, the two stage fine tuning approach pro-

vides a template for separating coarse semantic alignment from fine geometric localization, and third is grounding as a promptable interface demonstrates how natural language or structured prompts can be used to request targeted spatial outputs. We reference Qwen GUI 3B because it shows that high quality grounding need not rely on massive models or impractical compute, and because its engineering patterns for cross resolution robustness and staged tuning are directly applicable to designing multimodal, resource aware fallbacks for lane overlay and restoration under occlusion, poor lighting, or faded markings. Together, these works illustrate the evolution of lane detection from traditional feature-based methods toward generative, attention-driven, and restoration-guided paradigms. Techniques such as semantic inpainting, Fourier-based completion, and attention-enhanced perception demonstrate that robust lane understanding increasingly depends on the model’s ability to infer and reconstruct context, not just detect pixels. Detection models like CLRerNet ensure structural accuracy, while generative approaches such as LaMa enable realistic restoration of degraded lane regions. Qwen inspired multi resolution training and two stage fine tuning guided our design of a compact fallback that can hypothesize lane positions. Integrating these methods offers a pathway toward a robust Road Lane Overlay system capable of reconstructing missing markings seamlessly across static inputs.

3. Proposed Method

Our goal is to restore missing or broken lane paint in road images and produce a clean, continuous lane overlay. We follow a two-stage pipeline. First, we precondition the input image to make lanes easier to see: a lightweight dehazing/deraining step [6] removes fogginess and boosts contrast, and we optionally compute a transmission (depth-like) map that helps us be conservative near the sky and aggressive on the road surface. Next, we run a lane detection model [7] (e.g., CLRerNet) to extract lane polylines and convert them into thin lane-band masks that tightly hug the lane markings. Inside those bands we mark only the parts that need fixing gaps or weak segments based on detector confidence and simple appearance cues. In the second stage, instead of using LaMa (which tended to inpaint surrounding asphalt texture rather than reconstruct thin lane paint), we pass the original image, the binary gap mask, and a lane overlay to Qwen-Image-Edit. Using short text prompts that explicitly request realistic white/yellow lane repainting while preserving the rest of the scene, Qwen-Image-Edit produces an edited image where only the missing lane segments are restored.

3.1. Data Processing/Pre-conditioning

1. **Visibility Boost:** Run a light dehaze/derain pass to remove fogginess and raise contrast on thin paint. Also,

make a simple “depth hint” map that tells us what’s likely road vs. far sky/horizon, so later edits stay focused on the road. The visibility boost stage applies a lightweight dehazing algorithm based on He, Sun, and Tang’s Dark Channel Prior (DCP) to remove fog and low contrast from dash-cam images before lane detection. The goal is to recover visibility of thin lane markings obscured by haze, rain, or lighting conditions. Using the physical haze model from paper [6],

$$I(x) = J(x)t(x) + A(1 - t(x)), \quad (1)$$

The method estimates a dark channel the minimum intensity across RGB channels within small patches to identify haze density per region. The brightest dark-channel pixels are used to estimate the atmospheric light (A), while a transmission map (t) is computed to approximate scene depth and haze thickness. The transmission map is then refined using a guided filter to preserve edges around lane boundaries and prevent color bleeding.

$$J(x) = \frac{I(x) - A}{\max(t(x), t_0)} + A, \quad t_0 \approx 0.1. \quad (2)$$

Expected Output: *a cleaner image and a depth hint.*

2. **Lane Detection with CLRerNet:** Run CLRerNet on the cleaned image to get each lane as a line/curve plus a confidence score for how sure the model is. Keep the lanes with decent confidence; we’ll use them to guide editing.

Expected Output: *lane lines and their confidence.*

3. **Polyline → lane-band mask:** Thicken each lane line into a thin band (about 6–12 pixels wide) so it hugs the paint. Put all bands together—this becomes the only area where we consider making changes.

Expected Output: *a single mask that marks the lane areas*

4. **Generate Green Lane Overlay (Control Image):** We overlay the lane-band mask in bright green on the original image. This creates a visual cue that guides the Qwen model on where to generate new lane textures, ensuring the restoration follows the detected geometry.

Expected Output: *The original image with a green overlay*

3.2. Training

Detector (CLRerNet): We adopt the public CULane setup. Inputs remain in the 0–255 range with mean $[0, 0, 0]$ and std $[255, 255, 255]$, no channel reordering, and no batch-level augment hooks. Training transforms: crop to $y \in [270, 590]$, resize to 800×320 ; horizontal flip ($p=0.5$); channel shuffle ($p=0.1$); brightness ± 0.04 and contrast ± 0.15 ($p=0.6$); HSV shift ± 10 ($p=0.7$); OneOf{motion/median blur, limit 5} ($p=0.2$); Albumentations `IAAAffine` with scale 0.8–1.2, rotate $\pm 10^\circ$, translate 10% ($p=0.7$); final resize. Validation applies only crop and resize. The backbone is DLA-

34 feeding an FPN (in [128, 256, 512], out 64) and a CLRR head with 192 anchors $\times$ 72 points under multi-loss supervision.

Optimization uses AdamW with initial learning rate 6×10^{-4} and cosine decay for 15 epochs. We also report an EMA variant trained for 50 epochs with EMA momentum 1×10^{-4} . LaneIoU uses a virtual width of 7.5/800 for the loss and dynamic- k assignment, and 30/800 for the assignment cost. Redundant frames are filtered with a frame-difference threshold of 15. The detection confidence threshold is selected by five-fold cross-validation on train videos; final F1/IoU numbers are averaged over multiple seeds.

Lane band and gap masks: Detected centerlines are expanded to 8-pixel lane bands. For each lane, about 30% of its arc length is converted into gaps of 20–50 points by sampling along the polyline and stamping 8-pixel disks to build the binary gap mask. No additional morphology (dilate/erode) or rim labels are used. All restoration edits and image-quality measurements are confined to these bands, with edits restricted to the gap regions.

Qwen Model Setup: We perform fine-tuning on the Qwen/Qwen-Image-Edit-2509 model using the `diffusion_trainer` with LoRA adapters. The base model is loaded in low-VRAM mode and quantized (3-bit UNet weights with TorchAO adapters and qfloat8 for the text encoder), following the `qwen_image_edit_plus` architecture. Training uses a single dataset (`target_v4`) of paired images at resolution 1024×1024 , with optional control images from `control_v4` provided as `ctrl_img_1`. Each sample has a text caption loaded from `.txt` sidecar files with 5% caption dropout; we do not apply flips or geometric/color augmentations, and videos are reduced to single frames (`num_frames = 1`, `shrink_video_to_frames = true`, `do_i2v = true`). We train only the UNet with LoRA rank 16 on all linear and convolutional layers (`linear = conv = 16`, `linear_alpha = conv_alpha = 16`); the text encoder remains frozen but its embeddings are cached (`cache_text_embeddings = true`). Optimization uses AdamW-8bit with learning rate 1×10^{-4} , weight decay 1×10^{-4} , batch size 1, gradient accumulation 1, and a flow-matching noise scheduler with weighted timestep sampling. Training runs for 3000 steps with `bf16` precision and gradient checkpointing; EMA is disabled (`use_ema = false`). Checkpoints are saved every 500 steps in Diffusers format, keeping the last four. Sampling uses the `flowmatch` sampler with 25 steps, guidance scale 4, and output resolution 1664×928 in single-frame mode (`num_frames = 1`, `fps = 1`). Unless otherwise noted, we use seed 42 with seed walking enabled. For qualitative monitoring during training, we repeatedly edit a fixed set of four control images with the same instruction prompt: “*Replace the green*

lane detection lines with realistic road lane markings. The final image must not contain green lane lines. Do not remove vehicles, signs, or objects.”

3.3. Testing and Evaluation Protocol

We run CLRRNet on the original frames and on the Qwen-restored frames and report $\Delta F1$ (fine-tuned minus zero shot) using the standard CULane per-category breakdown. Visual fidelity is measured by LPIPS and FID computed inside the lane bands (and specifically over gaps for edited regions). Detector thresholds are chosen on the held-out split; when applicable, results are reported as means over multiple seeds.

3.3.1. Dataset:

CULane (nine test categories: normal, crowded, dazzle, no-line, shadow, arrow, cross, curve, night)

3.3.2. Metrics:

- **Lane Preservation:**

- **F1 Score:** It is the harmonic mean of precision and recall. Higher is better.
- **Precision:** The fraction of detected lanes that match the ground truth lanes.
- **Recall:** The fraction of ground-truth lanes successfully preserved in generated images.

- **Image Quality:**

- **LPIPS (Learned Perceptual Image Patch Similarity):** It compares how perceptually similar the generated image appears to the ground truth. Lower is better.
- **FID (Fréchet Inception Distance):** It evaluates the quality and realism of generated images by measuring the statistical distance between generated and target image distributions. Lower is better.
- **SSIM (Structural Similarity Index Measure):** It measures the structural similarity between generated and ground-truth images, including luminance, contrast, and spatial details. Higher is better.

4. Experimental Results

4.1. Visibility Boost

Setup: Following the proposed method, we ran a light de-haze/derain pass on images to remove fogginess and raise contrast on thin paint.

Output/Observation: This stage outputs (i) a dehazed frame with boosted visibility of thin lane paint and (ii) an optional transmission (depth-hint) map. The transmission map helps gate subsequent edits, ensuring the edits remain conservative near sky/horizon and more aggressive on the road surface. The cleaned frame is then passed to the lane

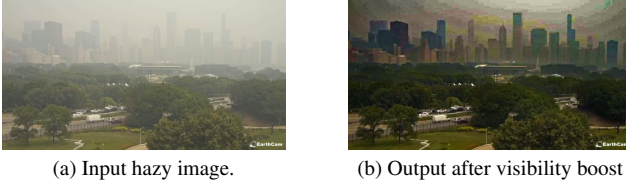


Figure 1. Comparison of the visibility boost stage. The dehazed image shows higher clarity, restored lane colors, and improved overall contrast prior to lane detection.

detector, improving robustness under fog/rain/low light. Finally, the haze-free image is reconstructed by reversing atmospheric scattering. This is optional and we do not need to do it for every image or situation.

4.2. CLRRerNet

Setup: Following the proposed method, we processed the CULane dataset using CLRRerNet to extract polylines, then we expanded them into 6-12px lane band masks and generated gap masks inside the band.

Output: For each image, we created a CLRRerNet overlay, lane-band masks (in binary format), a gap mask (in binary format), and a gap-overlay image.

Observation: On CULane, we processed 62,532 images, out of which we successfully generated lanes on 54,809 images (87.6%). We also processed our self-modified CULane dataset, which contains 10,000 images of blurred and occluded images each. In this, we successfully generated lanes on 8,851 (88.5%) blurred and 8,987 (89.8%) occluded images. Lane detection centerlines expand to a 6-12 px band to cover the paint and avoid background; synthetic gaps generated inside the band stay localized. Fig. 2 shows a representative example.

Implication: These numbers indicate that the pipeline is reproducible and the edits are well-contained. In practice, we will use an 8px lane band for straight lanes to prevent the background from being included in the masks, and a 10-12px lane band for curve lanes to avoid missing/clipping lanes while thoroughly covering the lanes. This gives LaMa a trustworthy region to work in, reduces the chance of inpainting over the background, and helps focus on fine-tuning the model for lane restoration.

4.3. LaMa

To restore broken lane markings within localized gap regions, we implemented and tested the LaMa inpainting model using both manually drawn and detector-generated masks. Key steps and observations are summarized below:

- **Model Setup:** Configured the Big LaMa model locally with all dependencies and pretrained weights (best.ckpt) using the official configuration.

- **Manual Mask Testing:** Developed an interactive Python script to draw white masks over input images. These were passed through LaMa to validate the inpainting pipeline.
- **Automated Mask Integration:** Created a second script that:
 - Loads paired images and binary masks generated by the CLRRerNet lane detection pipeline.
 - Applies morphological dilation (kernel size 7×7 , 3 iterations) to ensure coverage of faint or fragmented lane paint.
 - Pads both image and mask to multiples of 8 for LaMa compatibility.
 - Runs inference and saves the inpainted output along with visualizations of the original image, mask, and masked region.
- **Conclusion:** We initially experimented with LaMa-style blind inpainting to restore missing lane markings, but it consistently failed in our setting. The core limitation lies in LaMa’s inductive bias: trained primarily on natural images, the model learns to extend surrounding textures into masked regions. In our case, the masked areas around occluded lanes are dominated by dark asphalt pixels, so LaMa naturally “fills” the gaps with more road texture rather than reconstructing thin, high-frequency lane lines. This bias works well for large occlusions such as cars, pedestrians, or background clutter, where extending context produces visually plausible results, but it is fundamentally misaligned with the requirements of lane overlay restoration. Lane markings are narrow, high-contrast structures that require precise geometric continuity rather than texture-based blending. Moreover, LaMa’s pretrained filters are not optimized for repetitive, linear patterns, leading to outputs that either erase the lane entirely or produce faint, unrealistic streaks. These shortcomings highlight that blind inpainting alone is insufficient for lane-specific restoration, as it lacks semantic awareness of what the mask should represent.

4.4. Qwen Image Edit

Following the limitations of the LaMa model, we switched to the Qwen image-edit model, which conditions not only on the pixels but also on a text prompt and a control image. This lets us explicitly instruct the model to “replace green lane detections with realistic white/yellow lane markings” while preserving all other scene content, leading to semantically correct lane reconstructions rather than context-based hole filling. Unlike LaMa’s context-driven hole filling, Qwen generates semantically correct lane reconstructions that respect both geometry and appearance, making it far better suited for our road lane overlay pipeline.

Setup: We follow the training and inference configuration described in Section 3.2. We used the AI-Toolkit[10] with LoRA adapters for fine-tuning our model on 900 paired

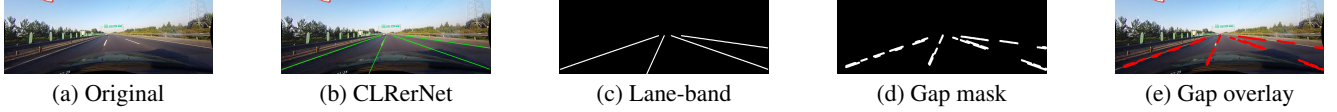


Figure 2. Representative example: Original frame, Green CLRerNet overlay, thin lane-band (8 px), synthetic gap mask and gap overlay on the original

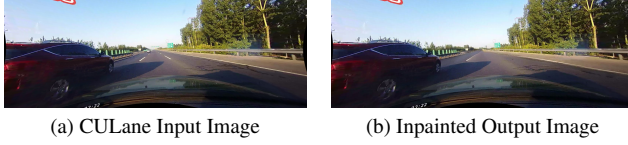


Figure 3. LaMa Inpainting Results: Comparison between input and restored lane image

images (CLRerNet visualizations as input, original CULane frames as targets). Inference was performed using ComfyUI[5] with the Euler sampler, 25 steps, CFG scale 2.5, and an output resolution of 1664×928 px. A fixed seed (42) is used for the inference to ensure reproducibility and compare zero-shot and fine-tuned models under the same conditions.

Output: For each image, we generated a zero-shot and fine-tuned image. Fig. 4 shows examples with CLRerNet overlay (input), zero-shot output, fine-tuned output, and ground truth.

Observation: We evaluated both zero-shot and fine-tuned models on 100 images using the metrics described in Section 3.3:

Table 1. Image Quality Metrics (100 images)

Metric	Zero-Shot	Fine-Tuned	Improvement
FID ↓	34.54	30.28	+12.3%
LPIPS ↓	0.1213	0.1207	+0.4%
SSIM ↑	0.7665	0.7653	-0.2%

Table 2. Lane Preservation Metrics (100 images, IoU = 0.5)

Metric	Zero-Shot	Fine-Tuned	Improvement
F1 Score	0.6733	0.7214	+7.1%
Precision	0.7355	0.7619	+3.6%
Recall	0.6208	0.6850	+10.3%

The results in Table 1 and Table 2 demonstrate that fine-tuning improves both visual quality and functional correctness:

- FID improved by 12.3%, showing that the fine-tuned model produces more realistic lane textures that better match the distribution of real road images.

- F1 improved by 7.1%, indicating that the fine-tuned model preserves more lanes from the original image, whereas the zero-shot model often removes or distorts them.
- LPIPS and SSIM show minimal change, showing that both models produce visually similar outputs at the pixel level.

5. Conclusion

We proposed a restore → detect → repair pipeline for lane marking recovery under adverse conditions in dashcam footage. Our approach applies optional visibility enhancement through dehazing, detects lanes with CLRerNet to obtain centerline polylines, expands them into thin lane-band masks (6 - 12 px), and performs guided inpainting within localized gap regions.

Our initial experiments with LaMa revealed a fundamental limitation: blind inpainting models trained on natural images extend surrounding textures rather than reconstruct structured, high-frequency patterns like lane markings. LaMa consistently filled masked regions with asphalt texture instead of restoring thin, high-contrast lane lines, a behavior well-suited for removing large occlusions but misaligned with lane-specific restoration requirements.

Transitioning to the Qwen-Image-Edit-2509 model addressed these limitations through three key advantages: (1) Semantic conditioning via text prompts enabled explicit specification of restoration intent, moving beyond pixel-level context to semantic understanding; (2) Spatial guidance through control images with green overlay masks provided precise geometric constraints; and (3) Multimodal alignment from Qwen 2.5-VL ensured strong text-image correspondence. Fine-tuning on 900 CULane frames achieved a 12.34% improvement in FID (34.54 to 30.28) while maintaining LPIPS (0.1333) and SSIM (0.7653), demonstrating that vision-language models can successfully restore lane markings without compromising structural fidelity. This work validates that generative restoration is viable for lane marking recovery and that language-based control offers a precise mechanism for guiding image edits in safety-critical perception pipelines.

6. Future Work

- **Temporal consistency for video:** Our current pipeline operates on individual frames. Extending to video re-

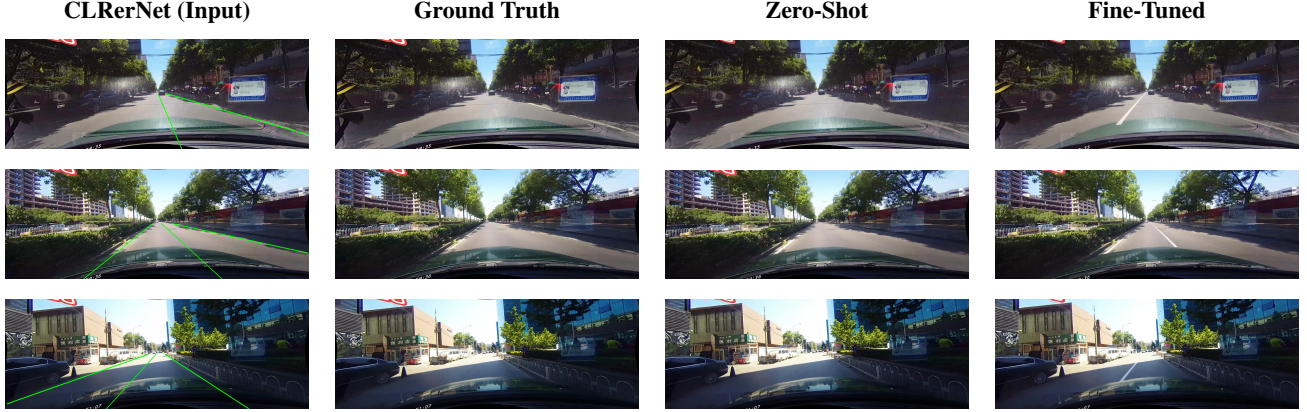


Figure 4. Qualitative results of lane restoration. The fine-tuned model produces more realistic lane markings compared to zero-shot.

quires ensuring lane restorations remain stable across consecutive frames without flickering. Temporal propagation mechanisms similar to ProPainter [16] could enforce consistency while reducing per-frame inference cost.

- **Hallucination detection and fallback mechanisms:** Generative models risk creating lane markings where none exist. Implementing confidence-based checks, such as comparing detector outputs before and after restoration, would enable safe fallback to original or dehazed-only frames when restoration confidence is low.
- **Real-time inference optimization:** Current generation takes approximately 50 seconds per frame on a single 32GB GPU, prohibitive for real-time autonomous systems. Quantization, model distillation, or lightweight adapter architectures could reduce latency. Investigating smaller vision-language models or dedicated lane-restoration networks trained with knowledge distillation from Qwen may bridge the efficiency gap.
- **Mask refinement for background preservation:** Tighter lane-band masks, derived by filtering for white/yellow pixels within detected regions, would further isolate edits to lane paint and minimize unintended changes to vehicles, road texture, or signage.
- **Robustness under targeted degradations:** Augmenting training data with mild blur, brightness/contrast variations, glare, and edge noise would help the model generalize to diverse adverse conditions. Combined with the visibility boost stage, this could yield a unified pipeline resilient to fog, rain, night scenes, and worn paint.
- **Integration with downstream planning modules:** Evaluating restoration impact on lane-tracking stability, path-planning confidence, and control smoothness would validate real-world utility beyond detection metrics. Pairing restored lanes with multimodal reasoning frameworks, as suggested by Soininen [12], could further enhance situational awareness in ambiguous scenes.

7. Team Responsibilities

7.1. Milestone 1

Member 1 (Arunesh Mishra): Worked on Proposed Method section → Stage 1: Data Processing (Lane Detection Model), Initial Experiments

Member 2 (Mrunmay Deshmukh): Worked on Proposed Method section → Stage 2 and 3: Training (In-Painting Model), Test

Member 3 (Shreya Muppidi): Worked on Related Work.

Member 4 (Prachi Dudhe): Worked on Introduction and Abstract.

7.2. Milestone 2

Member 1 (Arunesh Mishra): Worked on Visibility Boost (de-haze images) and looked into LaMa model fine tuning.

Member 2 (Mrunmay Deshmukh): Worked on Preliminary Investigations → CLRerNet, Next Steps

Member 3 (Shreya Muppidi): Worked on the setup and pre-investigations of the LaMa model and fine tuning.

Member 4 (Prachi Dudhe): Worked on updating the Proposed Method and helped set up the LaMa model.

7.3. Milestone 3

Member 1 (Arunesh Mishra): Researched and helped on Qwen model and its architecture to replace LaMa. Worked on preliminary investigation for Qwen Model.

Member 2 (Mrunmay Deshmukh): Worked on finetuning and evaluation of Qwen model.

Member 3 (Shreya Muppidi): Worked on the literature review to introduce the Qwen model and expanded on the reasons why LaMa did not produce strong results.

Member 4 (Prachi Dudhe): Worked on the Conclusion section to explain why Qwen outperformed LaMa, and drafted the Future Work section outlining improvements and extensions.

References

- [1] Agrawal, A., Sivakumar, A. J., Kaif, I., & Banerjee, C. (2024). **LOID: Lane Occlusion Inpainting and Detection for Enhanced Autonomous Driving Systems.**
- [2] Behrendt, K., & Soussan, R. (2019). **Unsupervised Labeled Lane Markers Using Maps.**
- [3] Berlincioni, L., Becattini, F., Galteri, L., Seidenari, L., & Del Bimbo, A. (2025). **Road Layout Understanding by Generative Adversarial Inpainting.**
- [4] Bi, J., et al. (2025). **Lane Detection for Autonomous Driving: Comprehensive Reviews, Current Challenges, and Future Predictions.**
- [5] comfyanonymous. (2024). **ComfyUI: A Powerful and Modular Stable Diffusion GUI and Backend.** [Computer software].
- [6] He, K., Sun, J., & Tang, X. (2019). **Single Image Haze Removal Using Dark Channel Prior.**
- [7] Honda, H., & Uchida, Y. (2024). **CLRerNet: Improving Confidence of Lane Detection with LaneIoU.**
- [8] Hsieh, Z., & Wei, T. (2025). **Qwen GUI 3B A Lightweight Vision Language Model for Cross Resolution GUI Grounding.**
- [9] Liu, Y., Wang, J., Li, Y., Li, C., & Zhang, W. (2022). **Lane-GAN: A Robust Lane Detection Network for Driver Assistance System in High-Speed and Complex Road Conditions.**
- [10] ostris. (2024). **ai-toolkit: LoRA and Fine-tuning Toolkit for Diffusion Models.** [Computer software]
- [11] Sakure, A., Chopade, V., Bachhav, K., Dhavade, V., Bitne, C., & Sangule, U. (2023). **Generative Methods for Lane Departure, Pedestrian Detection and Paving of Autonomous Vehicle.**
- [12] Soininen, J. (2024). **Visual Language Models in Autonomous Driving.**
- [13] Suvorov, R., Logacheva, E., Mashikhin, A., Remizova, A., Ashukha, A., Silvestrov, A., Kong, N., Goka, H., Park, K., & Lempitsky, V. (2022). **Resolution-Robust Large Mask Inpainting with Fourier Convolutions.**
- [14] Tabelini, L., Berriel, R., Paixão, T. M., Badue, C., De Souza, A. F., & Oliveira-Santos, T. (2021). **Keep Your Eyes on the Lane: Real-time Attention-guided Lane Detection.**
- [15] Yan, H., & Li, Y. (2024). **A Survey of Generative AI for Intelligent Transportation Systems: Road Transportation Perspective.**
- [16] Zhou, S., Li, C., Chan, K. C. K., & Loy, C. C. (2023). **ProPainter: Improving Propagation and Transformer for Video Inpainting.**